Go2 and Sensor

Go2 Setting

Go2 Setting

1) GO2 Package and Dependencies Install

Git

```
$ git clone https://github.com/unitreerobotics/unitree_ros2
```

Dependencies

```
$ sudo apt install ros-foxy-rmw-cyclonedds-cpp
$ sudo apt install ros-foxy-rosidl-generator-dds-idl
$ sudo apt install libyaml-cpp-dev
```

Dependencies

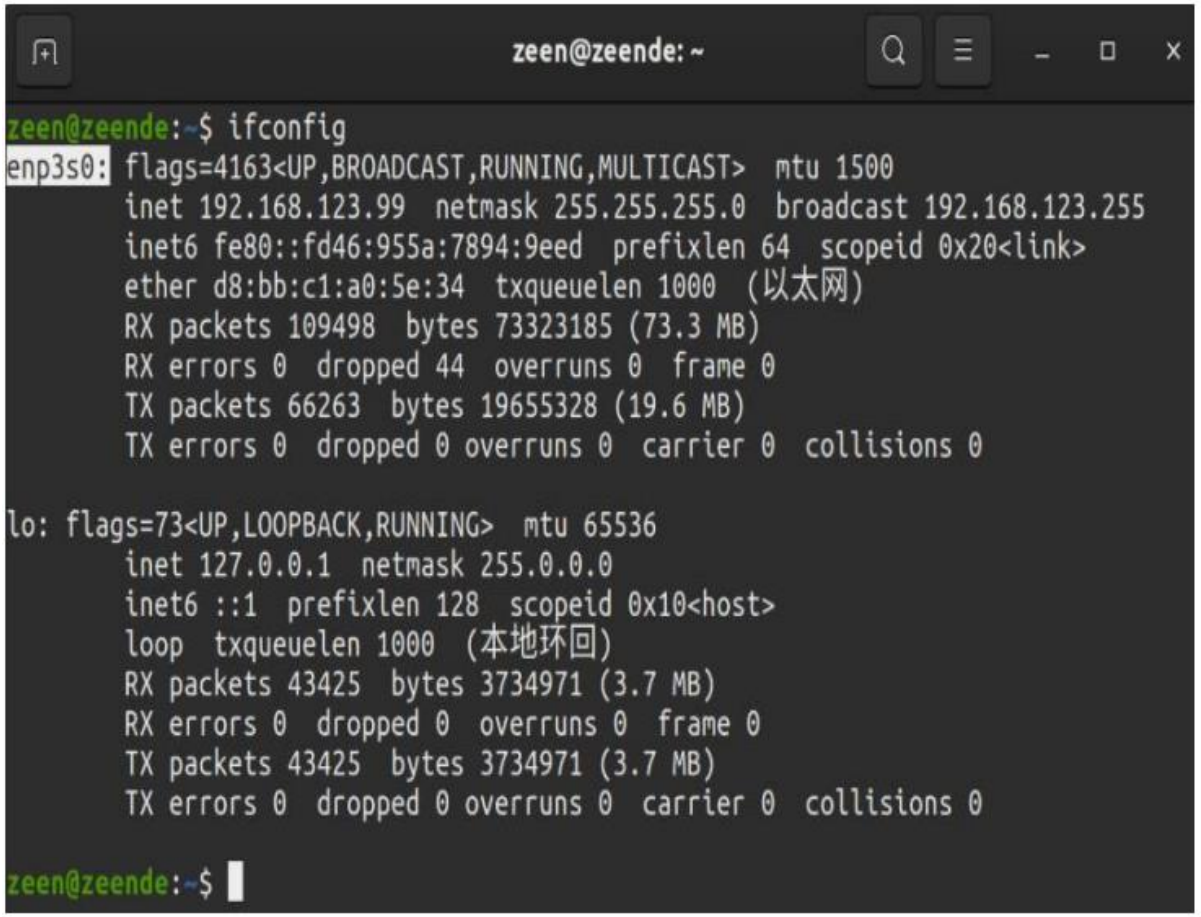
```
$ cd unitree_ros2/cyclonedds_ws/
$ source /opt/ros/humble/setup.bash
$ colcon build
```

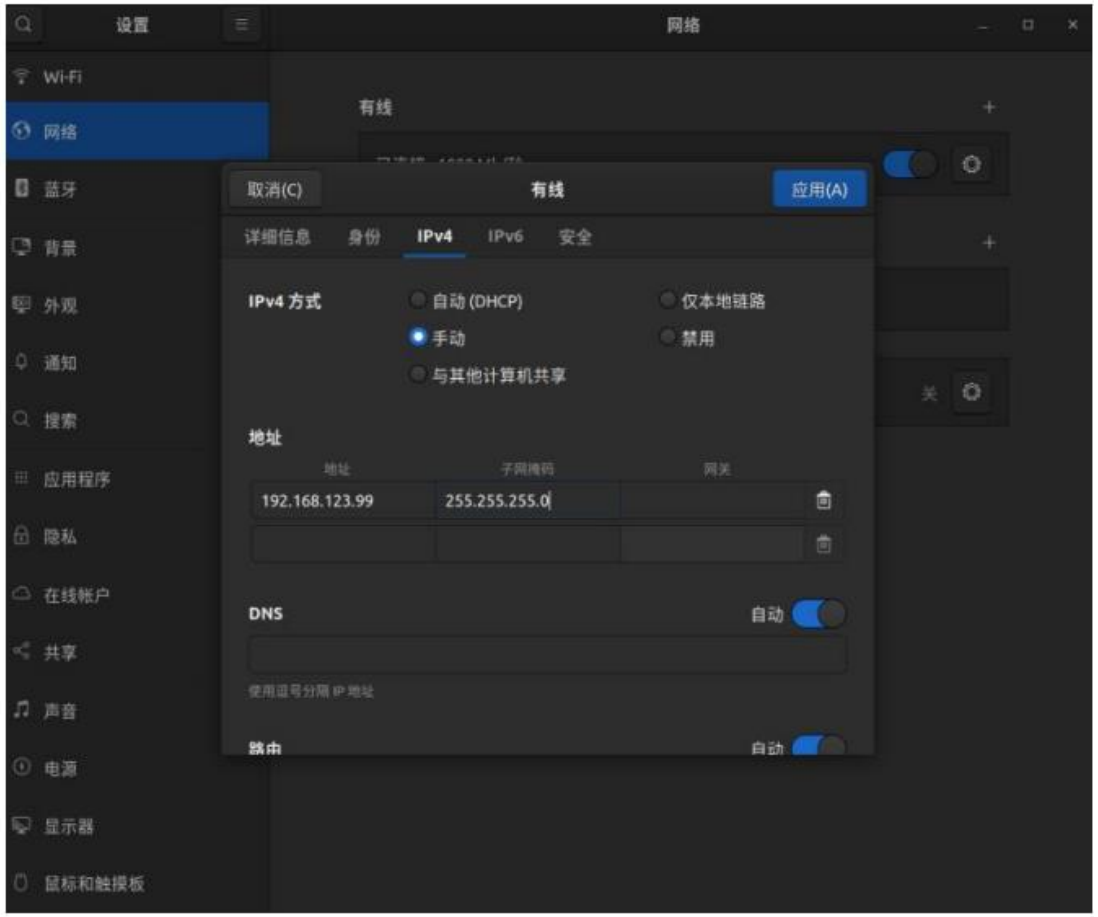
Install net-tools

```
$ sudo apt install net-tools
```

2) Network Setting

Network Setting





2) Network Setting

setup.sh Setting

```
$ gedit ~/unitree_ros2/setup.sh
```

```
#!/bin/bash
echo "Setup unitree ros2 environment"
source /opt/ros/foxy/setup.bash
source $HOME/unitree_ros2/cyclonedds_ws/install/setup.bash
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
export CYCLONEDDS_URI='<CycloneDDS><Domain><General><Interfaces>
                            <NetworkInterface name="enp3s0" priority="default" multicast="default" />
                        </Interfaces></General></Domain></CycloneDDS>'
```

Run

```
$ source ~/unitree_ros2/setup.sh
$ python3 ~/unitree_ros2/cmd_vel.py
$ ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

LiDAR Setting

LiDAR Setting

1) Velodyne Package Install

Installing ROS dependencies

```
$ sudo apt-get install ros-humble-velodyne
```

Installing the VLP16 driver

```
$ mkdir -p ~/sensor_ws/src
$ cd ~/sensor_ws/src
$ git clone https://github.com/ros-drivers/velodyne.git
$ cd ..
$ source /opt/ros/humble/setup.bash
$ rosdep install --from-paths src --ignore-src --rosdistro humble –y –r
$ colcon build
$ ros2 launch velodyne-all-nodes-VLP16-launch.py
```

LiDAR Setting

2) IP Setting

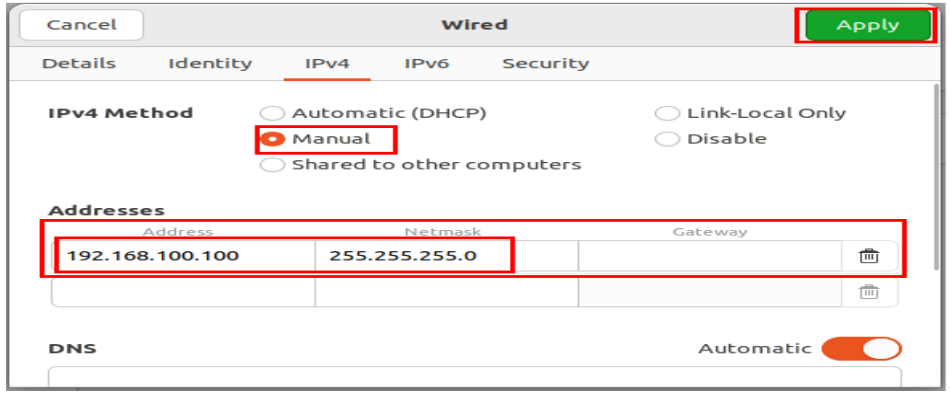
Installing wireshark

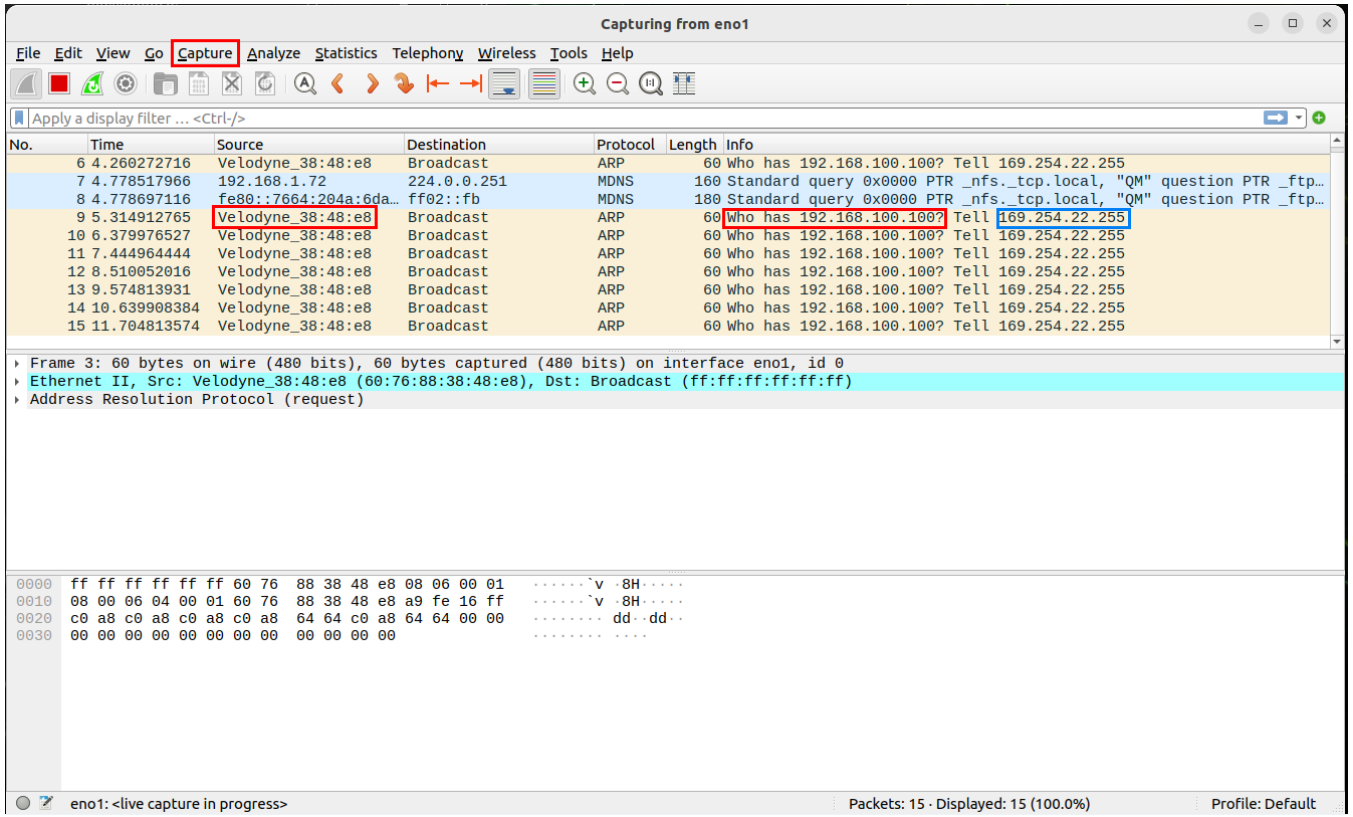
```
$ sudo apt install wireshark
```

Run wireshark

```
$ sudo wireshark
```

Static IP Setting





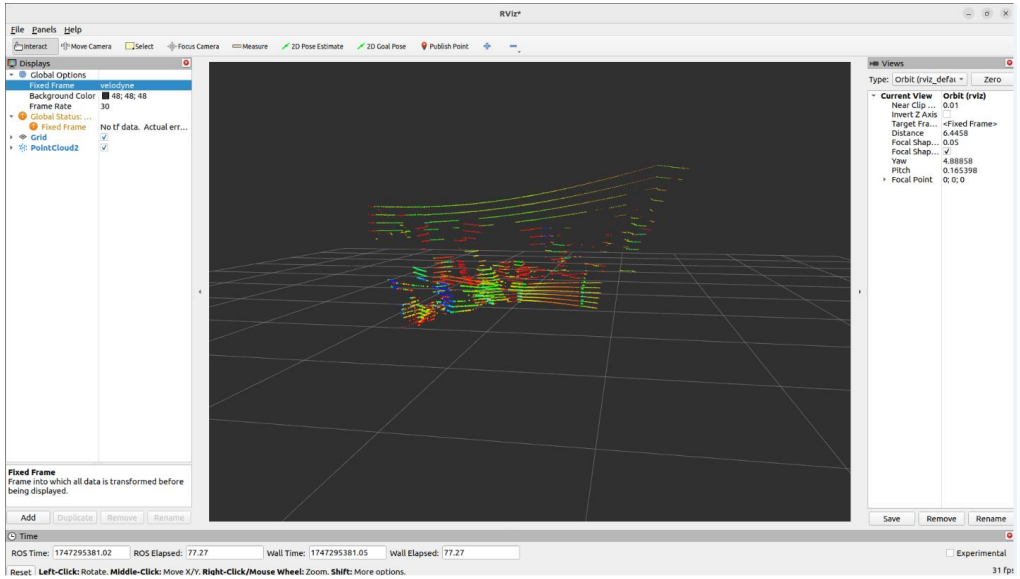
Adding Route

```
$ sudo route add 169.254.22.255 eno1 # eno1은 pc환경에 따라 변경 필요
```

LiDAR Setting

3) Launch

1. 인터넷 브라우저를 통한 LiDAR Parameter 접근
   - 인터넷 브라우저에 현재 Lidar IP 값을 입력하여 Data Port 및 IP 변경 및 확인 가능
2. ROS환경에서의 Lidar 구동을 위한 설정 파일(yaml) 수정
   - ~/sensor_ws/src/velodyne/velodyne_driver/config
   - gedit VLP16-velodyne_driver_node-params.yaml # params 수정
3. Build 및 실행
   - source /opt/ros/humble/setup.bash
   - cd ~/sensor _ws/
   - colcon build
   - source install/setup.bash
   - ros2 launch velodyne velodyne-all-nodes-VLP16-launch.py



Camera Setting

Camera Setting

1) RealSense Camera Setting

Installing Realsense-ROS

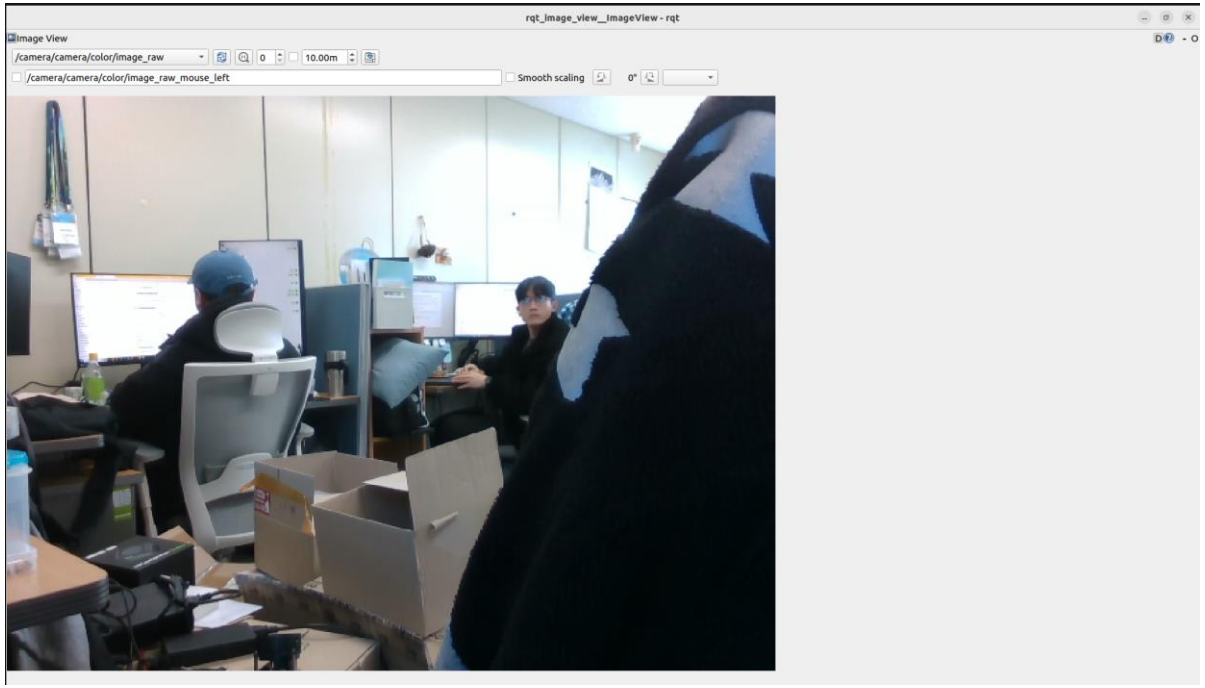
```
$ sudo apt-get install ros-humble-realsense2-camera
```

Strat Camera Node

```
$ ros2 launch realsense2_camera rs_launch.py
```

Strat Camera Node

```
$ ros2 launch realsense2_camera rs_launch.py
  pointcloude.enable:=true device_type:=d435
```



Camera Setting

2) USB Camera Setting

Installing USB_CAM

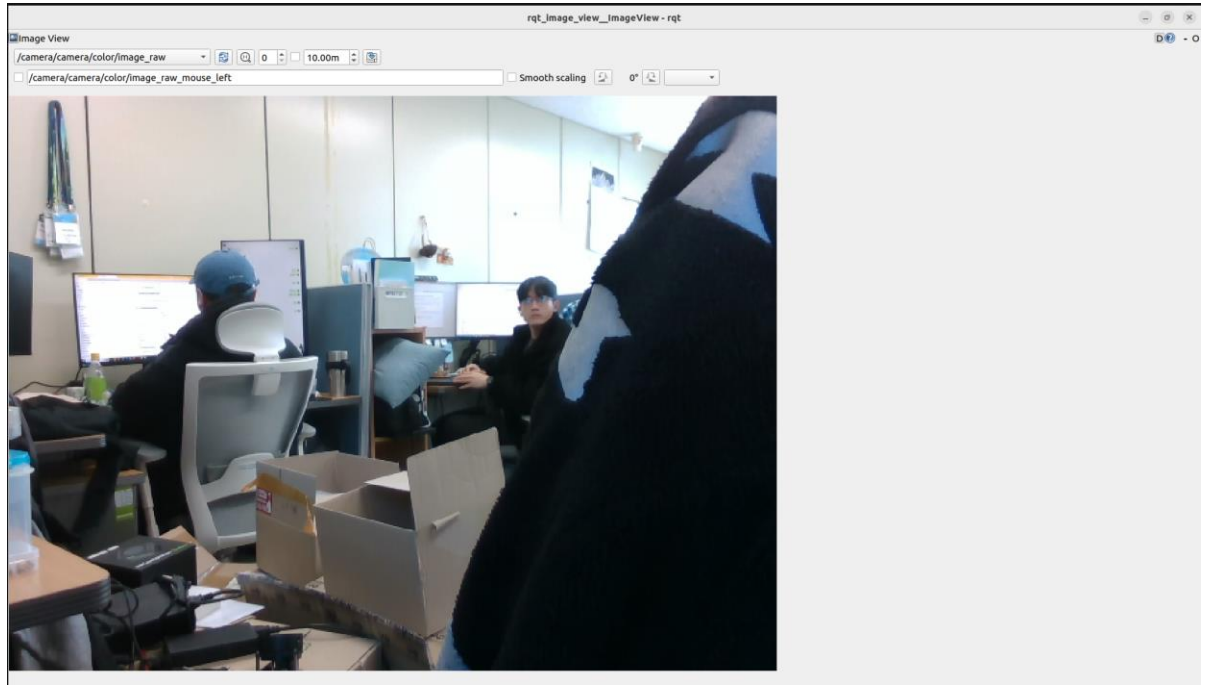
```
$ sudo apt-get install ros-humble-usb-cam
```

Strat Camera Node

```
$ ros2 run usb_cam usb_cam_node_exe
```

Run Rviz2

```
$ rviz2
```



ROS2 Camera Programing

ROS2 Camera Programing

1) Image Publish Node

Creating Package

```
$ cd ~/sensor_ws/src
$ ros2 pkg create --build-type ament_python camera_package
```

Creating Node

```
$ gedit ~/sensor_ws/src/camera_package/camera_package/image_publisher.py
```

Modify setup.py

```
$ gedit ~/sensor_ws/src/camera_package/setup.py
```

Build and run

```
$ cd ~/sensor_ws/
$ colcon build
$ source install/setup.bash
$ ros2 run camera_package image_publisher
```

ROS2 Camera Programing

1) Image Publish Node

```python
import cv2
import rclpy
from rclpy.node import Node

from sensor_msgs.msg import Image
from cv_bridge import CvBridge


class CameraPublisher(Node):

    def __init__(self):
        super().__init__('camera_publisher')

        self.publisher_ = self.create_publisher(Image, '/camera/image_raw', 10)

        self.bridge = CvBridge()

        self.cap = cv2.VideoCapture(0)

        self.timer = self.create_timer(1.0 / 30.0, self.timer_callback)

    def timer_callback(self):

        ret, frame = self.cap.read()

        if ret:
            msg = self.bridge.cv2_to_imgmsg(frame, encoding='bgr8')
            msg.header.stamp = self.get_clock().now().to_msg()
            msg.header.frame_id = 'camera_frame'

            self.publisher_.publish(msg)


def main():
    rclpy.init()
    node = CameraPublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

```

```python
from setuptools import find_packages, setup

package_name = 'camera_package'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='lab',
    maintainer_email='lab@todo.todo',
    description='TODO: Package description',
    license='TODO: License declaration',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
        'image_publisher = camera_package.image_publisher:main'
        ],
    },
)
```

2) Image Processing Node

Creating Node

```
$ gedit ~/sensor_ws/src/camera_package/camera_package/edge_detector.py
```

Modify setup.py

```
$ gedit ~/sensor_ws/src/camera_package/setup.py
```

Build and run

```
$ cd ~/sensor_ws/
$ colcon build
$ source install/setup.bash
$ ros2 run camera_package edge_detector
```

ROS2 Camera Programing

1) Image Publish Node

```python
import cv2
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge

class EdgeDetector(Node):

    def __init__(self):
        super().__init__('edge_detector')

        self.bridge = CvBridge()
        self.subscription = self.create_subscription(
            Image,
            '/camera/image_raw',
            self.image_callback,
            10)
        self.publisher_ = self.create_publisher(
            Image,
            '/camera/edge_image',
            10)
    def image_callback(self, msg):
        # ROS Image -> OpenCV 이미지
        frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')

        # grayscale 변환
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # Canny edge detection
        edges = cv2.Canny(gray, 100, 200)

        # OpenCV 이미지 -> ROS Image
        edge_msg = self.bridge.cv2_to_imgmsg(edges, encoding='mono8')
        edge_msg.header = msg.header

        self.publisher_.publish(edge_msg)

def main():
    rclpy.init()
    node = EdgeDetector()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

```python
from setuptools import find_packages, setup

package_name = 'camera_package'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='lab',
    maintainer_email='lab@todo.todo',
    description='TODO: Package description',
    license='TODO: License declaration',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
        'image_publisher = camera_package.image_publisher:main',
        'edge_detector = camera_package.edge_detector:main',
        ],
    },
)
```